



AI Support Request and Management System Comprehensive Security Assessment Report

Version Number – 2.0

Date 20/2/2026

Warning

This report contains confidential and privileged information about the security status of AI Support Request and Management System. The information is intended for the private use of the AI Support Request and Management System Development team and authorized personnel. Access to this information by unauthorized personnel may allow them to compromise your information technology infrastructure or it could be used as a resource to attackers for further attacking analysis. Therefore, Ethiopian Artificial Intelligence Cyber Operation and system security division recommends keeping this information confidential and do not distribute it without the consent or written approval.

This evaluation reveals all relevant vulnerabilities known up to the date of this report and the capability of our testing team. As new vulnerabilities and new security threats emerge daily, it is suggested that the security assessment be conducted regularly

Table of Contents

AI Support Request and Management System Comprehensive Security Assessment Report ...	1
Acronyms.....	4
1. Introduction.....	5
1.1. Executive Summary.....	5
1.2. Summary of Findings.....	6
1.3. Project Objective	7
1.4. Project scope.....	7
2. Audit Findings.....	8
2.1. Detailed Security Audit Findings	8
1. Broken Access Control.....	8
2. Privilege Escalation	12
3. Missing Data Transfer Objects (DTOs) / Information Exposure.....	15
4. Insecure Local Storage.....	18
5. Improper JWT Token Usage	20

Acronyms

Terminology	Definition
HTTPS	Secure Hypertext Transfer Protocol
HTTP	Hypertext Transfer Protocol
EAI-COSS	Ethiopian Artificial Intelligence Cyber Operation and System Security Division
SSL	Secure Sockets Layer
TLS	Transport Layer Security
URL	Uniform Resource Locator
OWASP	Open Web Application Security Project
API	Application Programming Interface
JS	Javascript
RCE	Remote Code Execution
JSON	Javascript Object Notation
DTO	Data Transfer Object
CWE	Common weakness and exposure

1. Introduction

1.1. Executive Summary

Ethiopian Artificial Intelligence Cyber Operation and System Security Division has performed vulnerability assessment and penetration testing on AI Support Request and Management System. EAI-COSS has conducted the security assessment in a manner that simulated malicious actors engaged in an attack against the web app using different security vulnerability technical tools and best practices and measured the overall security status of the web application.

The assessment identified multiple critical authorization and data-handling flaws that allow low-privileged users to perform administrative actions, escalate privileges, access sensitive data, and maintain unauthorized sessions.

The most severe weaknesses stem from:

- Missing role-based access control enforcement
- Excessive data exposure from APIs
- Insecure token lifecycle design
- Sensitive data stored client-side

These issues indicate systemic backend authorization architecture flaws rather than isolated programming mistakes. If exploited by a malicious user, the platform could be fully compromised, including unauthorized administrative access and sensitive data exposure.

Immediate remediation is strongly recommended.

1.2. Summary of Findings

Types of Vulnerabilities	Level of Risks		
	CRITICAL	HIGH	MEDIUM
Broken Access Control	✓		
Privilege Escalation	✓		
Missing Data Transfer Objects		✓	
Insecure Local Storage			✓
Improper JWT Token Usage			✓
Total	2	1	2

1.3. Project Objective

The main objective of this vulnerability assessment and penetration testing is to identify potential security vulnerabilities, assess their impact on the application's integrity, confidentiality, and availability, and provide actionable technical recommendations to remediate them. This process aims to enhance the overall security posture of the application and protect it against potential threats and exploitation.

1.4. Project scope

Scope: Authentication, Authorization, Session Management, API Security, Client-Side Storage

- <http://196.188.240.103:4037>

2. Audit Findings

2.1. Detailed Security Audit Findings

The following show the details of the vulnerabilities identified during the security assessment process.

1. Broken Access Control

Severity Level: Critical

CVSS Score: 9.1 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

Endpoints:

- POST /api/users
- PUT /api/users/{user_id}
- GET /api/roles
- GET /api/permissions
- /api/notifications

Vulnerability Description

The application fails to implement functional-level access control. While the system verifies that a user possesses a valid JSON Web Token (JWT), it does not validate the user's authorization to perform specific actions on restricted endpoints.

Technical Explanation

Authorization is only verified at the authentication layer (token presence). There is no secondary check to ensure the user's assigned role or permissions correspond to the requested administrative action. Consequently, an attacker with a low-privileged account (e.g., "testone") can directly interact with administrative APIs.

Proof of Concept

1. Authenticate as a low-privileged user to obtain a Bearer token.
2. Intercept a request to an administrative endpoint, such as `GET /api/roles`.
3. Observe that the server returns a `200 OK` or `201 Created` response containing sensitive data or confirming the action, despite the user's low-privileged status.

[illegible]

Impact

A low-privileged user can perform any administrative action, including creating new users, editing permissions, and viewing sensitive system-wide notifications. This represents a total loss of confidentiality, integrity, and availability.

Countermeasures

- **Implement Role-Based Access Control (RBAC):** Enforce server-side checks for every request to ensure the user's role has the explicit permission required for the specific endpoint and HTTP method.
- **Deny by Default:** Implement a security model where access is denied unless specifically allowed for a user role.

2. Privilege Escalation

Severity Level: Critical

CVSS Score: 9.0 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

Endpoint: `POST /api/users`

Vulnerability Description The application allows low-privileged users to create new accounts and assign them high-level roles and permissions.

Technical Explanation

The user creation endpoint allows low-privileged users to assign arbitrary role identifiers when creating new accounts.

Because the backend does not validate role assignment permissions, attackers can create accounts with administrative privileges.

1. Log in as a low-privileged user and capture administrative `role_id` and `permission_id` values from the API responses.
2. Submit a `POST` request to `/api/users` with the `role_ids` field populated with the captured administrative UUIDs.
3. Observe that the application successfully creates the "testfour" user with the elevated privileges.

```

1 HTTP/1.1 201 Created
2 Content-Security-Policy: default-src
  'self';base-uri 'self';font-src 'self'
  https: data:;form-action
  'self';frame-ancestors 'self';img-src
  'self' data:;object-src
  'none';script-src
  'self';script-src-attr 'none';style-src
  'self' https:
  'unsafe-inline';upgrade-insecure-reques
  ts
3 Cross-Origin-Opener-Policy: same-origin
4 Cross-Origin-Resource-Policy:
  cross-origin
5 Origin-Agent-Cluster: ?1
6 Referrer-Policy: no-referrer
7 Strict-Transport-Security:
  max-age=31536000; includeSubDomains
8 X-Content-Type-Options: nosniff
9 X-DNS-Prefetch-Control: off
10 X-Download-Options: noopen
11 X-Frame-Options: SAMEORIGIN
12 X-Permitted-Cross-Domain-Policies: none
13 X-XSS-Protection: 0
14 Access-Control-Allow-Origin:
  http://196.188.240.103:4037
15 Vary: Origin

```

```
NTZkZC1jMDFLTQ1YjYkOTY3MCOxODU1ZjVhYzI3NDEiLCJhY3Rpb24iOiJlcGRhdGUiLCJyZXRhbnRkZSI6Imh1bW
WfUx3Jlc291cmNlcyJ9LHsicGVybnZlc2lvdWZ3ZC1jY3Rpb24iOiJlcGRhdGUiLCJyZXRhbnRkZSI6Imh1bW
I2ZSI6ImFjdGlvbiI6ImRlbgV0ZSI6ImNlc291cmNlIjoiaHVhYV5fcmVzb3VvY2VzIn0seyJwZXJtaXNzaW9uX2l
kIjoimWVlYmYnNjktMmM1ZS00NDE3LTkyZWItMGQwOTAzOTY2MjY2IiwiaWNOaW9uIjoiy3JlYXRlIiwicmVzb3Vv
Y2UiOiJodW1hbG9yZXNvdXJjZXMiOjE3NDEiLCJhY3Rpb24iOiJlcGRhdGUiLCJyZXRhbnRkZSI6Imh1bW
DZlMDYwbnZnNGIiLCJhY3Rpb24iOiJyZXRhbnRkZSI6Imh1bWZ3VvY2VzIn0seyJwZXJtaXNzaW9uX2l
pbeyJybnZlc2lvdWZ3ZC1jY3Rpb24iOiJlcGRhdGUiLCJyZXRhbnRkZSI6Imh1bWZ3VvY2VzIn0seyJwZXJtaXNzaW9uX2l
gTWfUx3Jlc291cmNlcyJ9LHsicGVybnZlc2lvdWZ3ZC1jY3Rpb24iOiJlcGRhdGUiLCJyZXRhbnRkZSI6Imh1bW
qWp2J9Bi7T6vMUqCEpFBFjNv7Fb5LAoFc-0
```

7 content-type: application/json

8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/145.0.0.0 Safari/537.36

9 Origin: http://196.188.240.103:4037

0 Referer: http://196.188.240.103:4037/

2 Connection: keep-alive

```
3 {
4   "full_name": "testfour",
5   "email": "testfour@gmail.com",
6   "phone_number": "+251911111131",
7   "user_type_id": "49e261a5-f319-4685-8978-ef1339f1a0eb",
8   "role_ids": [
9     "830c900f-377a-44b0-ad1d-11e3f67fe4d7",
10    "5343b953-eca3-4b43-bd5b-58c9a80fca64",
11    "8e0e5df5-3d4b-4188-bcb4-578aa9bcc356",
12    "2e1282d6-bbf1-4889-aa16-134d64716c24",
13    "2f843673-6363-4643-803b-2787e54a9f8f",
14    "f23e308f-a183-419f-830f-cf7e44966716",
15    "35ff0f7a-40e8-4624-aa0a-71fafca44b9d"
16  ],
17   "institute_id": "1cce43ed-9b6b-4e5b-8b11-0d1c496aad5b",
18   "user_position_id": "7971070d-5eal-4e23-a08d-e0be5982c45a"
19 }
```

15 Vary: Origin

16 Access-Control-Allow-Credentials: true

17 Content-Type: application/json; charset=utf-8

18 Content-Length: 715

19 ETag: W/"2cb-LKwRhl+LufIX1YXBWxgtuZcURA"

20 Date: Thu, 19 Feb 2026 20:39:40 GMT

21 Connection: keep-alive

22 Keep-Alive: timeout=5

```
23
24 {
25   "success": true,
26   "message": "User registered globally (no roles assigned yet)",
27   "data": {
28     "user_id": "5cd5f994-ab61-458a-a6fb-1edd30db9340",
29     "full_name": "testfour",
30     "email": "testfour@gmail.com",
31     "password": "$2b$10$BI/hkOIIM0ZokReQ/44I7ubotdjH8N09L..c60XaMkaYrsQ8aDny",
32     "phone_number": "+251911111131",
33     "user_type_id": "49e261a5-f319-4685-8978-ef1339f1a0eb",
34     "institute_id": "1cce43ed-9b6b-4e5b-8b11-0d1c496aad5b",
35     "user_position_id": "7971070d-5eal-4e23-a08d-e0be5982c45a",
36     "hierarchy_node_id": null,
37     "is_first_logged_in": true,
38     "is_active": true,
39     "created_at": "2026-02-19T20:39:38.264Z",
40     "updated_at": "2026-02-19T20:39:38.264Z",
41     "position": null,
42     "internal_node_id": null,
43     "profile_image": null,
44     "last_login_at": null,
45     "password_changed_at": null
46   }
47 }
```

Impact

This allows an attacker to bypass all security restrictions by creating an "all-powerful" account, leading to full system compromise.

Countermeasures

- **Restrict User Management:** Only allow users with a "System Administrator" role to access user-creation and role-assignment endpoints.
- **Validate Assignments:** Ensure the server validates that the requester has the authority to assign the specific roles included in the request payload.

3. Missing Data Transfer Objects (DTOs) / Information Exposure

Severity Level: High

CVSS Score: 7.5 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

Endpoint: Application-wide API responses (e.g., Login, User Update)

Vulnerability Description

The application lacks a Data Transfer Object (DTO) layer, causing it to return raw database entities in API responses.

Technical Explanation

By directly returning database records, the application reveals sensitive fields that should never be exposed to the client-side. This includes internal database schemas, role/permission UUIDs, and even sensitive credentials.

Proof of Concept (Step-by-Step Reproduction)

1. Perform a standard login or update a user profile.
2. Inspect the JSON response body.
3. Note the presence of the `password` field containing a bcrypt hash (e.g., starting with `$2b$10$`) and internal `role_id` UUIDs.

```
POST /api/auth/login HTTP/1.1
Host: 196.188.240.103:4038
Content-Length: 49
Accept-Language: en-US,en;q=0.9
accept: application/json
content-type: application/json
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Origin: http://196.188.240.103:4037
Referer: http://196.188.240.103:4037/
Accept-Encoding: gzip, deflate, br
Connection: keep-alive

{
  "email": "admin@gmail.com",
  "password": "password"
}
```

```
{
  "permission_id": "298c919e-a0a5-4ece-8d05-ea99fc39744e",
  "action": "view_own",
  "resource": "dashboard"
},
{
  "permission_id": "72d37695-a271-4144-8118-fb99f2ca8c37",
  "action": "view_all",
  "resource": "dashboard"
}
],
"roles": [
  {
    "role_id": "b5b1f1b5-0d41-4bf1-9043-5274a125e3f8",
    "name": "admin"
  }
],
"metrics": [
],
"phone_number": "251911000001",
"profile_image": null,
"is_first_logged_in": false,
"project_roles": [
],
"internal_project_roles": [
]
}
```


- **Use DTOs:** Map database entities to specific DTO classes that only contain the fields required by the UI before sending the response.
- **Field Filtering:** Explicitly exclude sensitive fields (like `password`, `deleted_at`, or internal metadata) from serialization.

4. Insecure Local Storage

Severity Level: Medium

CVSS Score: 5.7 (CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:N/A:N)

Endpoint: Browser Local Storage

Vulnerability Description

The application stores sensitive user profile information, including email, roles, permissions, and phone numbers, in the browser's Local Storage.

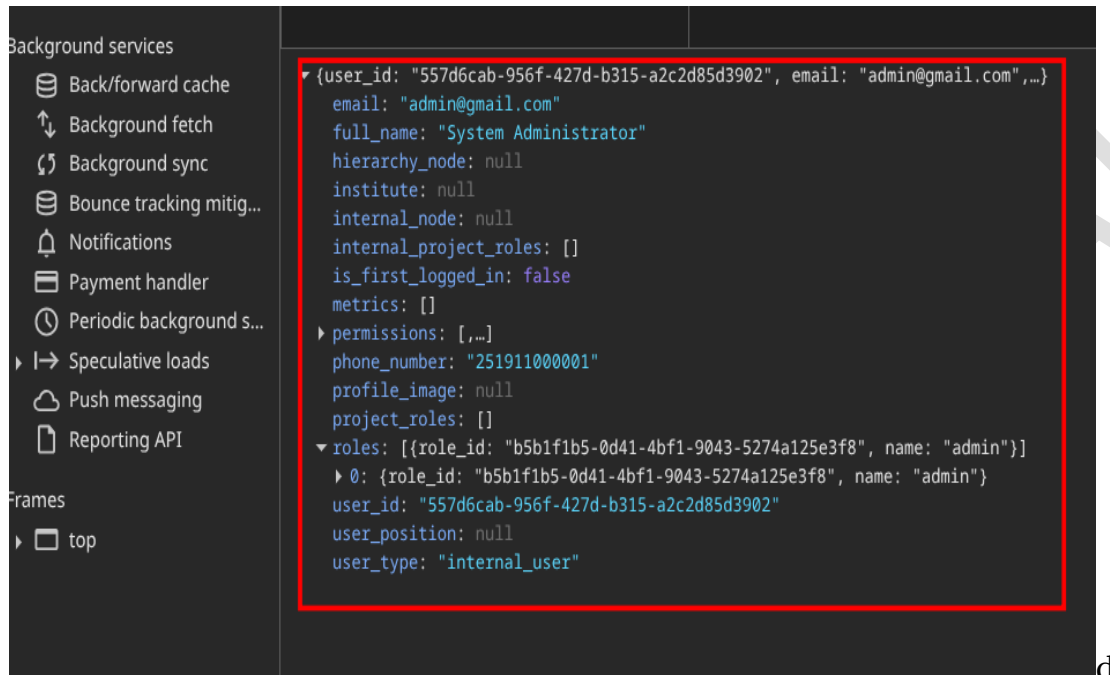
Technical Explanation

Local Storage is accessible via JavaScript on the same origin. Storing sensitive PII (Personally Identifiable Information) and authorization details here makes them vulnerable to extraction.

Proof of Concept (Step-by-Step Reproduction)

1. Log into the application.

2. Open Browser Developer Tools (F12) and navigate to the "Application" tab.
3. Select "Local Storage" and observe the stored JSON object containing `user_id`, `email`, `full_name`, and `phone_number`.



Impact

In the event of a Cross-Site Scripting (XSS) vulnerability, an attacker can programmatically steal this sensitive data. It also exposes user data on shared or public computers.

Countermeasures

- **Minimize Storage:** Avoid storing PII or sensitive authorization details in Local Storage.
- **Use Session Storage:** If data is needed temporarily, use `sessionStorage`, which is cleared when the tab is closed.

5. Improper JWT Token Usage

Severity Level: Medium

CVSS Score: 3.7 (CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N)

Endpoint: Authentication System / JWT Implementation

Vulnerability Description

The application issues access tokens with an excessively long expiration period and lacks a refresh token mechanism.

Technical Explanation

The current JWT **exp** (expiration) claim is set to 1 hour. Best practices for high-security applications typically recommend a shorter window (10–15 minutes). Furthermore, without a refresh token, users are forced to re-authenticate manually once the token expires, or developers are tempted to set even longer expirations to avoid "disrupting" the user experience.

Proof of Concept (Step-by-Step Reproduction)

1. Authenticate and decode the JWT (e.g., using jwt.io).
2. Inspect the **iat** (issued at) and **exp** (expiration) claims.
3. Observe the 1-hour delta between the two timestamps.

```
    "resource": "human_resources"
  }
],
"roles": [
  {
    "role_id": "830c900f-377a-44b0-ad1d-11e3f67fe4d7",
    "name": "Project Manager2"
  }
],
"metrics": [],
"iat": 1771532560,
"exp": 1771536160
}
```

Impact

- If an access token is compromised, the attacker has a wide 60-minute window to use it before it becomes invalid. The lack of a refresh mechanism limits the ability to implement secure, short-lived sessions.

Countermeasures

- **Shorten Token Lifespan:** Reduce the Access Token expiration to 15 minutes or less.
- **Implement Refresh Tokens:** Use a secure, [HttpOnly](#), [SameSite=Strict](#) cookie to store a refresh token, allowing the application to silently request new access tokens.